

# ADVANCE PROBLEM PROGRAMMING

## SLO 2: Abstraction & Specification

### 1. Write a Java interface and implement it in a class.

An interface in Java is used to achieve abstraction. It contains method declarations that are implemented by a class.

```
interface Animal {  
    void sound();  
}  
  
class Dog implements Animal {  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

#### Output:

Dog barks

Here, Animal is an interface containing the abstract method sound(). The Dog class implements the interface and provides the implementation of sound().

### 2. Demonstrate abstraction using access modifiers.

Access modifiers can be used to hide internal data and provide controlled access through public methods.

```
class Student {  
    private int marks;  
  
    public void setMarks(int marks) {  
        this.marks = marks;  
    }  
  
    public int getMarks() {  
        return marks;  
    }  
}  
  
public class Main {
```

```
public static void main(String[] args) {  
    Student s = new Student();  
  
    s.setMarks(85);  
  
    System.out.println("Student Marks: " + s.getMarks());  
}  
}
```

### Output:

Student Marks: 85

Here, marks is declared as private, so it cannot be accessed directly from outside the class. The public methods setMarks() and getMarks() provide controlled access to the data. This demonstrates abstraction by hiding the internal implementation.

### 3. Give a real-world analogy for abstraction and relate it to Java classes.

A car is a real-world example of abstraction. A driver uses the steering wheel, accelerator, brake, and gear to operate the car without knowing the internal working of the engine, fuel injection, or transmission system.

Similarly, in Java, a class can hide its internal implementation and provide only the necessary methods to the user.

For example:

```
class Car {  
    private int speed;  
  
    public void accelerate() {  
        speed = speed + 10;  
    }  
  
    public void displaySpeed() {  
        System.out.println("Speed: " + speed + " km/h");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car c = new Car();  
  
        c.accelerate();  
        c.displaySpeed();  
    }  
}
```

### Output:

Speed: 10 km/h

In this example, the user does not need to know how the speed is internally calculated. The Car class hides the internal data and provides simple methods such as `accelerate()` and `displaySpeed()`. This is similar to how a real car hides its complex internal mechanisms from the driver.